SwiftDrop

Scope Division for 10 Team Members

Parcel Delivery & Courier Management System | IN3101 Enterprise Application Development | Group Project

Prepared from: the assignment guideline, the System Architecture Proposal v2 (eight-module modular monolith) and the UI design set (35 screens).

How to read this document

- Section 1 explains how the scope was split and shows each member's workload at a glance.
- Section 2 holds the per-member scopes: UI screens, backend, database, tests and demo role.
- Section 3 maps every UI screen to its owner; Section 4 lists screens still missing from the UI set.
- Sections 5-7 cover shared agreements, dependencies/timeline and the common deliverables.

1. Approach and workload summary

Each member owns a vertical slice: the React screens, Spring Boot logic, database tables and tests for their area. Nobody is frontend-only or backend-only, so every member can demo and explain their own contribution (required by the marking scheme). Slices follow the eight architecture modules, so members work in parallel against agreed module interfaces.

Principles used to balance the work:

- Every work item is given points (1 point is roughly half a day of effort for one developer), covering UI, backend, database and tests. Every member totals exactly 30 points (300 in total).
- Heavy modules were split (Parcel & Pricing across Members 3 and 4; Dispatch & Delivery across Members 6 and 7; Merchant Integration across Members 8 and 9). Light modules were paired (User Management with Reporting; Notifications with Deployment).
- UI screens are spread so each member owns between 2 and 7 screens, with heavier backend work offsetting lighter UI and vice versa.
- Bonus items (AI Assistant, Kubernetes) are extra and not counted in the 30 points; they go to Members 5 and 10.

Workload at a glance

#	Slice	Architecture module(s)	Member name	Points	Bonus
1	Identity, Access & Shared UI Shell	Identity & Access		30	-
2	Admin Management, Merchant Approval & Reports	User Management + Reporting & Operations		30	-
3	Delivery Booking, Zones & Pricing	Parcel & Pricing (booking, fee) + Zones & Configuration		30	-
4	Parcel Lifecycle, History & Customer Deliveries	Parcel & Pricing (status, history, receipts)		30	-
5	Public Tracking, Real-time Updates & AI Assistant	tracking + real-time status updates		30	Yes
6	Dispatch & Courier Workload	Dispatch & Delivery (dispatcher side)		30	-
7	Courier Mobile App & Proof of Delivery	Dispatch & Delivery (courier side)		30	-
8	Merchant Portal & API Key Management	Merchant Integration (portal side)		30	-
9	Merchant Order API & Webhooks	Merchant Integration (API & webhooks)		30	-
10	Events, Notifications & Deployment	Notifications & Events + deployment & quality		30	Yes

Names are left blank so the group can fill them in or draw lots.

2. Member scopes

Member 1: Identity, Access & Shared UI Shell

Name: _____

Architecture modules: Module 1 – Identity & Access

Workload: 30 points

UI screens

- Login page (incl. Continue with Google)
- Customer registration page
- Shared app shell: sidebars/top bars per role, route guards, role-based redirect
- System state kit: 404, 403 Access Denied, Session Expired, General Error, Loading, Empty, Success, Failure, Confirmation dialog

Backend

- JWT issue/refresh/logout, BCrypt password hashing
- Google OAuth 2.0 login flow
- Role-based access control (Customer, Merchant, Courier, Dispatcher/Admin) as reusable guards

Database: users (credentials), refresh_tokens, roles

Tests: Login, token validation, role checks (e.g. courier cannot read another courier's deliveries)

Demo role: Customer login incl. Google login; role-based access

Work item	Points
Login + customer registration screens with validation	4
JWT auth, refresh, logout, password hashing	6
Google OAuth 2.0 flow (backend + button)	5
RBAC: roles, method-level guards, React route guards, redirect	5
Shared UI shell + system-state components	5
Auth & authorization tests	3
DB design: users, roles, refresh tokens	2
Total	30

Member 2: Admin Management, Merchant Approval & Reports

Name: _____

Architecture modules: Module 2 – User Management; Module 8 – Reporting & Operations

Workload: 30 points

UI screens

- Merchant registration form + 'Registration Submitted' screen
- Admin: User Management
- Admin: Merchant Management (list) and Merchant detail (BrightMart view)
- Admin: Operational Reports
- Customer: Profile & Account

Backend

- User/role management APIs (create, deactivate, change role)
- Merchant registration with pending/approved/rejected workflow + admin approve/reject
- Customer profile endpoints
- Reporting queries: deliveries per day, successful deliveries, failed-delivery rate

Database: user_profiles, merchants (status fields), report views/queries

Tests: Approval workflow, unapproved merchant blocked, report accuracy

Demo role: Admin approves a merchant; reports dashboard

Work item	Points
Merchant registration + submitted screens	3
Admin User Management screen + APIs	5
Customer Profile & Account screen + APIs	3
Admin Merchant Management + detail screens	4
Merchant approval workflow (states, API, notification hook)	5
Operational Reports screen (charts, date filter)	4
Reporting queries / views	3
Tests (approval, reports)	3
Total	30

Member 3: Delivery Booking, Zones & Pricing

Name: _____

Architecture modules: Module 3 – Parcel & Pricing (booking, fee); Module 7 – Zones & Configuration

Workload: 30 points

UI screens

- Book a Delivery – Step 1: Pickup & Drop-off
- Book a Delivery – Step 2: Parcel Details (size, weight, parcel type) – UI MISSING
- Book a Delivery – Step 3: Review, fee estimate & Confirm + confirmation – UI MISSING
- Admin: Pricing Rules
- Admin: Service Areas
- Admin: System Settings

Backend

- Create delivery request, generate public tracking number (SD + digits)
- Fee estimate API using Strategy pattern (zone, weight, size, service type)
- Zone / service-area lookup and validation; pricing rules CRUD; system settings

Database: parcels (creation fields), addresses, zones, pricing_rules, system_settings

Tests: Fee calculation per zone/weight, address and size validation

Demo role: Customer books a delivery and sees fee before confirming

Work item	Points
Booking wizard Step 1 (existing UI)	3
Booking Step 2 Parcel Details (design + build)	3
Booking Step 3 Review/Confirm + fee preview	4
Parcel creation API + tracking number generator	5
Pricing engine (Strategy) + fee estimate API	5
Admin Pricing Rules, Service Areas, System Settings screens + CRUD	7
Tests	3
Total	30

Member 4: Parcel Lifecycle, History & Customer Deliveries

Name: _____

Architecture modules: Module 3 – Parcel & Pricing (status, history, receipts)

Workload: 30 points

UI screens

- Customer: Dashboard
- Customer: My Deliveries / delivery history
- Customer: Delivery Tracking (logged-in detail view)
- Delivery receipt download
- Admin: Parcels (list + detail drawer)

Backend

- Status state machine: Created, Picked Up, In Transit, Out for Delivery, Delivered/Failed with valid-transition checks
- Status history recording; publish a status-change event on every transition
- Receipt PDF generation
- Customer parcel queries (own parcels only); admin parcel search/filter

Database: status_history, parcel status fields

Tests: Valid/invalid transitions, history recorded, event published, customer sees only own parcels

Demo role: Status changes visible in history; receipt download

Work item	Points
Status state machine + transition validation	6
Status history + event publishing	4
Customer Dashboard	4
My Deliveries + history with filters	4
Customer delivery detail/tracking view	3
Receipt PDF generation + download	3
Admin Parcels screen + APIs	3
Tests	3
Total	30

Member 5: Public Tracking, Real-time Updates & AI Assistant

Name: _____

Architecture modules: Module 3 – tracking; Module 6 – real-time status updates

Workload: 30 points (+ bonus: AI Delivery Assistant (+5 marks))

UI screens

- Public landing page
- Public Parcel Tracking (search)
- Tracking results: In Transit, Delivered, Not Found
- Admin: Deliveries Tracking (live monitor + details drawer)
- Live-update hooks used by customer, merchant and admin screens

Backend

- Public tracking endpoint by tracking number (no personal data exposed, rate-limited)
- WebSocket/SSE channel pushing status changes (consumes broker events)
- Bonus: AI delivery assistant ("Where is my parcel?", "How much to send 2 kg to Kandy?") via backend APIs / MCP tools

Database: Reads parcels and status_history (no new core tables)

Tests: Public endpoint leaks no personal data, real-time push on status change

Demo role: Live tracking page updating in real time

Work item	Points
Landing page	4
Public tracking search + result states	5
Public tracking API (no PII, rate limiting)	4
WebSocket/SSE real-time channel	7
Live-update client hooks for other roles	3
Admin Deliveries Tracking screen	4
Tests	3
Total	30

Member 6: Dispatch & Courier Workload

Name: _____

Architecture modules: Module 4 – Dispatch & Delivery (dispatcher side)

Workload: 30 points

UI screens

- Admin/Dispatcher: Operations Overview (3 design variants)
- Unassigned parcels list + Assign action (single and bulk)
- Admin: Couriers (list, workload, courier detail drawer, Add Courier)
- Failed deliveries queue with Retry / reassign

Backend

- List unassigned parcels, assign/reassign courier
- Courier workload counts and availability (Online / On Break / Offline)
- Courier registration/profile (vehicle, service areas)
- Failed-delivery retry flow

Database: couriers, assignments

Tests: Assignment rules, workload counts, only dispatcher/admin can assign

Demo role: Dispatcher assigns a parcel to a courier

Work item	Points
Operations Overview dashboard	5
Unassigned parcels list + assign UI	5
Assignment API + rules	5
Courier management screens (list, drawer, add)	5
Failed deliveries queue + retry/reassign	4
Courier entity & availability APIs	3
Tests	3
Total	30

Member 7: Courier Mobile App & Proof of Delivery

Name: _____

Architecture modules: Module 4 – Dispatch & Delivery (courier side)

Workload: 30 points

UI screens

- Courier: Today's Deliveries
- Courier: Delivery Details
- Courier: Scan Parcel (QR/barcode + manual entry)
- Courier: Completed Deliveries
- Courier: Update Status screen – UI MISSING
- Courier: Proof of Delivery (recipient name, photo) – UI MISSING
- Courier: Failed Delivery reason form – UI MISSING

Backend

- Courier-only list of own deliveries
- Status updates calling Member 4's state machine
- Proof upload and storage (recipient name, photo)
- Failure reasons (required on Failed)

Database: delivery_proofs, failure_reasons

Tests: Courier sees only own deliveries, proof saved, failure reason required

Demo role: Courier updates status, uploads proof, marks a failed delivery

Work item	Points
Today's Deliveries screen + API	3
Delivery Details screen	3
Parcel Scanner screen	4
Status Update screen (design + build)	4
Proof of Delivery (UI + upload + storage)	5
Failed delivery reason form + records	4
Completed Deliveries + courier profile	3
Ownership checks + tests	4
Total	30

Member 8: Merchant Portal & API Key Management

Name: _____

Architecture modules: Module 5 – Merchant Integration (portal side)

Workload: 30 points

UI screens

- Merchant: Dashboard
- Merchant: Orders
- Merchant: Order Details
- Merchant: Business Profile
- Merchant: API Credentials panel (API Integration page – key ID, secret, generate/revoke)

Backend

- Dashboard and order queries (all orders and delivery states)
- Merchant profile endpoints
- API key generation, hashing, rotation and revocation (secret shown once)
- Gate: only approved merchants may use portal/API

Database: merchants (profile), api_keys / oauth_clients, merchant_orders link

Tests: Unapproved merchant cannot use the API, key revocation works

Demo role: Merchant dashboard with orders and generated API key

Work item	Points
Merchant Dashboard	4
Orders list	4
Order Details	3
Business Profile	3
Dashboard/order/profile backend queries	5
API key management UI + backend	6
Approved-only gating + data model	2
Tests	3
Total	30

Member 9: Merchant Order API & Webhooks

Name: _____

Architecture modules: Module 5 – Merchant Integration (API & webhooks)

Workload: 30 points

UI screens

- Merchant: Delivery Updates (webhook URL settings, event/delivery log; 2 design variants)
- Merchant: API Integration page – status, setup steps, docs and test environment links

Backend

- Versioned REST order API (POST /v1/orders, get order, get status) with validation
- API-key auth filter + OAuth 2.0 client-credentials token endpoint
- Webhook dispatcher consuming status events, HMAC-signed payloads
- Retry with exponential backoff and failed-attempt log
- OpenAPI/Swagger docs + sandbox test webhook

Database: webhook_subscriptions, webhook_attempts

Tests: Order creation via API, webhook sent on status change, retry on failure

Demo role: Merchant creates order via API and receives webhook

Work item	Points
Versioned order API	6
API-key auth + OAuth2 client credentials	6
Delivery Updates screens + Integration page	4
Webhook dispatcher + HMAC signing	5
Retry/backoff + failed log	4
OpenAPI docs + sandbox	2
Tests incl. retry	3
Total	30

Member 10: Events, Notifications & Deployment

Name: _____

Architecture modules: Module 6 – Notifications & Events; deployment & quality

Workload: 30 points (+ bonus: Kubernetes deployment (+5 marks))

UI screens

- Customer: Notifications page (history + preferences)
- Notification bell / unread badge in all top bars

Backend

- RabbitMQ topology and status-change event schema (agreed with Members 4 and 9)
- Notification consumer: email / mock SMS, templates, error handling & dead-letter queue
- Notification log API
- Dockerfiles + docker-compose (frontend, backend, PostgreSQL, RabbitMQ)
- CI pipeline, env-var config, DB migrations & seed data
- Aggregated coverage report and test report

Database: notifications log, migrations/seed setup

Tests: End-to-end event flow: status change, event, notification; coverage summary

Demo role: Notification triggered by a status change; docker-compose up

Work item	Points
RabbitMQ topology + event schema	5
Notification consumer (email/mock SMS, DLQ)	6
Notifications page + bell + preferences + log API	5
Dockerfiles + docker-compose	5
CI, env config, migrations, seed data	4
Coverage/test report + event-flow integration test	5
Total	30

3. UI screen ownership

Every screen in the UI folder, mapped to the member who builds it (front end and its APIs). Members 1 and 10 also supply shared components used everywhere.

Screen	Owner (Member #)
Public landing page	5
Login page	1
Registration - Customer	1
Registration - Merchant	2
Merchant registration submitted	2
Public parcel tracking (search)	5
Tracking result - In Transit / Delivered / Not Found	5
System states: 404, 403, session expired, error, loading, empty, success, failure, confirm dialog	1
Customer - Dashboard	4
Customer - Book a Delivery (Step 1)	3
Customer - My Deliveries (Delivery Dashboard)	4
Customer - Delivery Tracking	4
Customer - Notifications	10
Customer - Profile & Account	2
Courier - Today's Deliveries (Delivery Dashboard)	7
Courier - Delivery Details	7
Courier - Parcel Scanner	7
Courier - Completed Deliveries	7
Courier folder also holds Courier Management, Parcel Management and Delivery Tracking; these are admin views, see Admin rows	6 / 4 / 5
Merchant - Dashboard	8
Merchant - Orders	8
Merchant - Order Details	8
Merchant - API Integration	8 (credentials) / 9 (status, docs)
Merchant - Delivery Updates (2 variants)	9
Merchant - Business Profile	8
Admin - Operations Dashboard (3 variants)	6
Admin - Parcel Management	4
Admin - Courier Management	6
Admin - Delivery Tracking	5
Admin - Merchant Management (2 variants)	2
Admin - Merchant Dashboard (merchant detail)	2
Admin - Operational Reports	2
Admin - Pricing Rules	3
Admin - Service Areas	3

Screen	Owner (Member #)
Admin - System Settings	3
Admin - User Management	2

4. Screens missing from the UI set

The assignment guideline notes flag these gaps. Each needs a quick design (Figma or sketch) by its owner before build; the effort is already included in that member's points.

Missing screen	Owner	Note
Parcel details (size, weight, parcel type) - booking Step 2	3	Parcel type must be included
Review & confirm with fee estimate - booking Step 3	3	Fee shown before confirming
Courier status update screen	7	Triggers state machine
Courier proof of delivery (recipient name / photo)	7	Photo upload
Courier failed delivery reason form	7	Reason is mandatory
Customer delivery receipt view / download	4	PDF
Forgot password / password reset (optional)	1	Nice to have

5. Shared agreements (settle in week 1)

Agreement	Owner	Agreed with
Tech stack, repo structure, branching, coding style, API base path (/api/v1)	Member 10	Everyone
Token claims and role names	Member 1	Everyone
Parcel status values and allowed transitions	Member 4	3, 5, 6, 7, 9, 10
Status-change event schema (RabbitMQ)	Member 10	4, 5, 9
Database naming, migrations and seed data	Member 10	Everyone
Docker/compose service names and environment variables	Member 10	Everyone
Shared UI design tokens, layout shell and component library (SwiftDrop green theme)	Member 1	Everyone
Error response format and centralised exception handler	Member 1	Everyone
Pricing interface used by booking	Member 3	4, 9

6. Dependencies and suggested timeline

Phase	Focus	Key hand-offs
Week 1	Agreements, repo, Docker skeleton, auth skeleton, DB migrations	M10 delivers compose + CI; M1 delivers login + role guards; M4 publishes status enum and event contract
Weeks 2-3	Core slices built against stubs	M3 parcel creation feeds M4 state machine; M2 approval feeds M8 gating; M6 assignment pairs with M7 courier list
Weeks 4-5	Integration: events, real-time, webhooks, notifications	M4 events feed M5 live push, M9 webhooks, M10 notifications
Week 6	Testing, coverage, security review, bug fixes	M10 aggregates coverage; every member covers own tests
Week 7	Documentation, demo rehearsal, submission	All members rehearse their demo segment

Critical path: Member 1 (auth) and Member 4 (state machine and events) unblock most other slices, so they should deliver a working first version in weeks 1-2. Others can use mock data until then.

7. Common deliverables

Deliverable	Lead	Contribution from others
Source repository + README setup instructions	Member 10	Each member documents their module's endpoints
Design documentation (use cases, ER, architecture, integration, sequence, patterns)	Member 2 (compile)	Already drafted; each member checks their own module's diagrams
Test reports and coverage summary	Member 10	Each member supplies unit/integration results
Contribution sheet	Member 1	Each member lists their responsibilities from Section 2

Deliverable	Lead	Contribution from others
Presentation and live demo	Member 5 (host)	Each member demos their own part (see Demo role)

Demo flow: Customer login (M1), booking with fee (M3), merchant API order + webhook (M8, M9), dispatcher assigns (M6), courier updates status + proof (M7), live tracking (M5), notification (M10), reports (M2).

Rules for everyone: own your slice's UI, API, tables and tests; validate all input; use environment variables for config; never log personal data; use the shared error handler.